

Electrical Cabinet Library

Technical Documentation & User Manual v2026.1

Contents

- 1 Introduction 2
 - 1.1 Styling 2
 - 1.2 Anchors 2
 - 1.3 Wiring 2
- 2 Layout Infrastructure 2
 - 2.0.1 Cabinet (cabinet) 2
 - 2.0.2 DIN Rails (dinrail) 3
 - 2.0.3 Wire Ducts (wireduct) 3
 - 2.1 Component Library 4
 - 2.1.1 Protective Devices (mcb) 4
 - 2.1.2 Power Supply (psu) 4
 - 2.1.3 Modular Boxes (generic_box) 4
 - 2.2 Advanced Wiring Techniques 5
 - 2.2.1 The Projection Principle 5
 - 2.2.2 Terminal Strips (terminal_strip) 5
 - 2.2.3 Bridging (bridge) 6

1 Introduction

This library provides a modular framework for creating technical 2D electrical cabinet layouts using Typst and CeTZ. It bridges the gap between manual CAD drawing and automated schematic generation.

Throughout this documentation, you will find visual examples alongside code snippets. The light gray boxes show the rendered circuit diagrams, with the corresponding Typst code displayed underneath.

1.1 Styling

All parameters can be set explicitly or inherited from the global style. For example, setting `wire: (stroke: blue)` in the circuit style will make all wires blue by default. Individual wires can still override this by specifying their own stroke color.

These are the parameters that every component has:

- `label`: The text label displayed on the component.
- `text-size`: The font size of the label.
- `width` and `height`: The dimensions of the component. If not specified, some components will calculate their width based on the number of terminals (e.g., `psu`).
- `fill`: The background color of the component. By default, components have a light gray fill, but this can be customized for better visual distinction.

1.2 Anchors

- **Definition:** Anchors are predefined connection points on components, such as `p1` and `p2` for terminals. They allow for precise wire connections without needing to specify exact coordinates.
- **Exceptions:** The generic boxes and all components that use them (like PLCs) have dynamic anchors based on their pin configuration. A pin can either be accessed by its name given in the pin array (e.g., `PLC1.CAN-L`) or by its position in the group (e.g., `PLC1.11` for the first pin on the left side).

1.3 Wiring

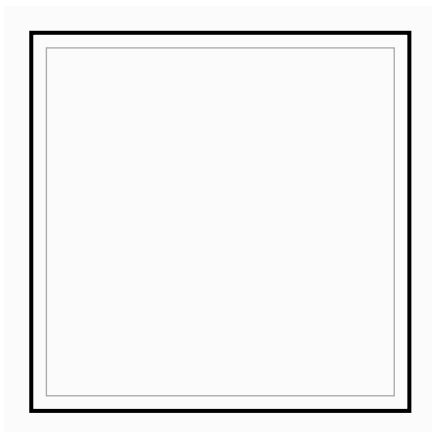
- **Syntax:** It follows exactly the format of the zap library. If you need more information on how to use it, please refer to the zap documentation.

2 Layout Infrastructure

2.0.1 Cabinet (`cabinet`)

The container for the entire assembly. It generates a grid-based coordinate system using "slots".

- **Key Anchors:** `slot-1` to `slot-n` represent horizontal mounting levels.
- **Usage:** `#cabinet("main", (0,0), rows: 5, width: 15, height: 25).`



Visual Example:

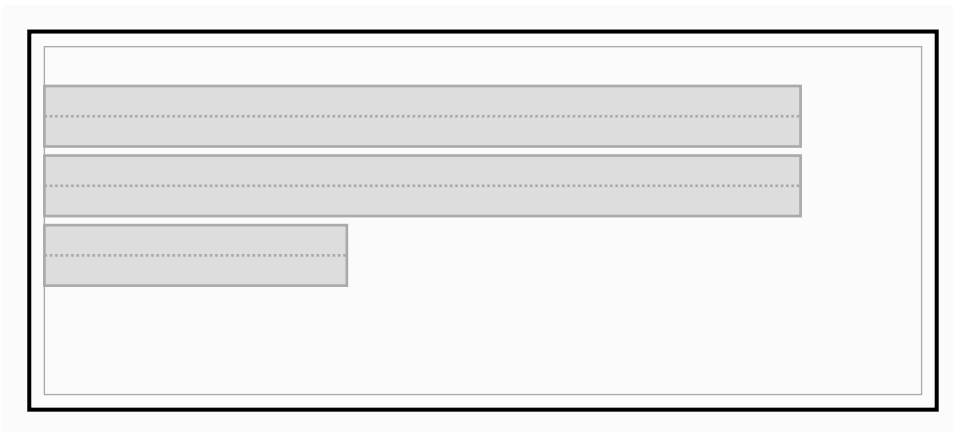
Code:

```
#circuit(debug: false, {  
  cabinet("main", (0, 0), rows: 4, width: 5, height: 5)  
})
```

2.0.2 DIN Rails (dinrail)

The physical mounting surface for components.

- **Tip:** Always place rails at cabinet slot anchors for perfect alignment.
- **Orientation:** Use orientation: "vertical" for vertical rails.
- **Example:** #dinrail("r1", "main.slot-2").



Visual Example:

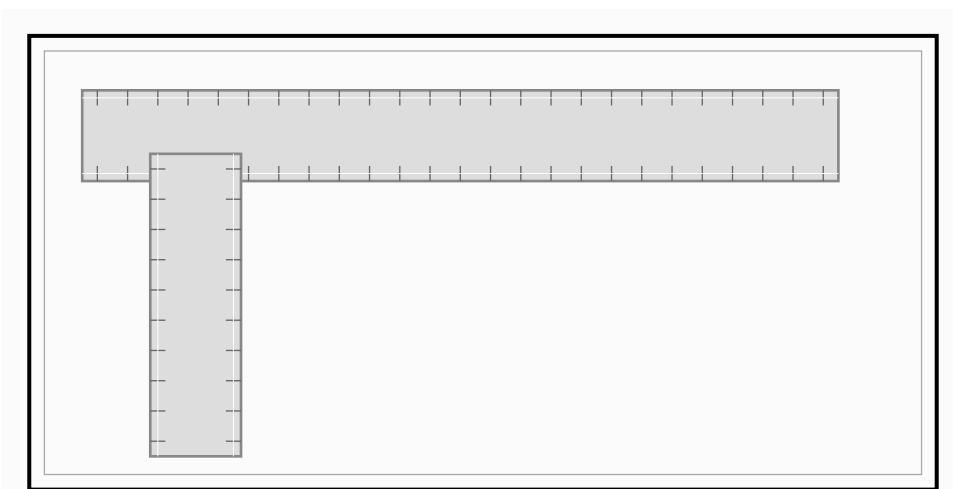
Code:

```
#circuit(debug: false, {  
  cabinet("main", (0, 0), rows: 4, width: 12, height: 5)  
  dinrail("rail1", "main.slot-1", length: 10)  
  dinrail("rail2", "main.slot-2", length: 10)  
  dinrail("rail3", "main.slot-3", length: 4, orientation: "vertical")  
})
```

2.0.3 Wire Ducts (wireduct)

Used for cable management.

Setting orientation: "vertical" rotates the duct correctly while maintaining logical anchors.



Visual Example:

Code:

```
#circuit(debug: false, {  
  cabinet("main", (0, 0), rows: 4, width: 12, height: 5)  
  wireduct("rail1", (rel:(0.5, 0), to:("main.slot-1")), length: 10)
```

```
wireduct("rail2", (rel:(2, -2), to:("main.slot-3")), length: 4, orientation: "vertical")
})
```

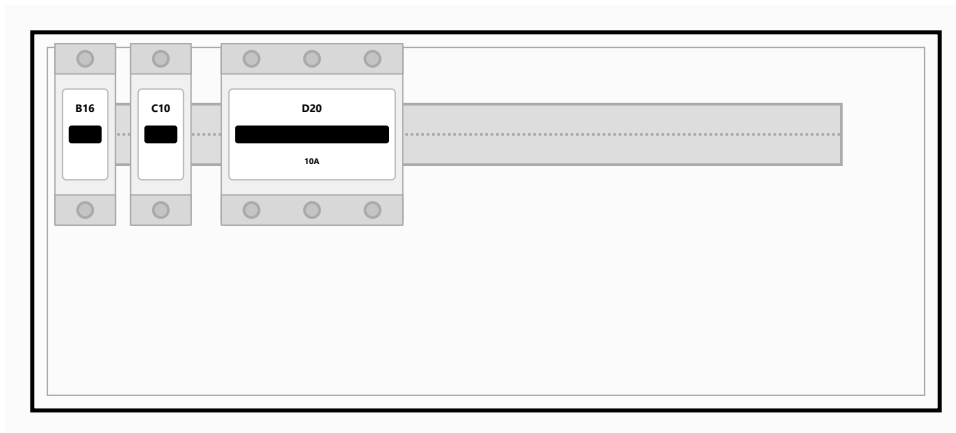
The first creates a horizontal cable duct, the second a vertical one for organizing wiring.

2.1 Component Library

2.1.1 Protective Devices (mcb)

The Miniature Circuit Breaker represents a single-pole protection unit. Also supports multi-pole configurations by specifying the phases parameter.

- **Anchors:** p1 (Line/Top), p2 (Load/Bottom) or added a _n suffix for multi-pole (e.g., p1_2 for the second pole).
- **Example:** #mcb("F1", "rail1.west", label: "B16").



Visual Example:

Code:

```
#circuit(debug: false, {
  cabinet("main", (0, 0), rows: 3, width: 12, height: 5)
  dinrail("rail1", (rel: (0.5, 0), to:("main.slot-1")), length: 10)
  mcb("F1", (rel: (0,0), to:("rail1.west")), label: "B16")
  mcb("F2", (rel: (1,0), to:("rail1.west")), label: "C10")
  mcb("F3", (rel: (3,0), to:("rail1.west")), label: "D20", phases: 3, label-bottom: "10A")
})
```

2.1.2 Power Supply (psu)

A dynamic module that calculates its own width based on terminal count.

- **Terminals:** Supports l_pins (AC input) and configurable L+/M output pairs.
- **Logic:** Terminal colors are automatically applied (Red for +, Blue for -).

Code Example:

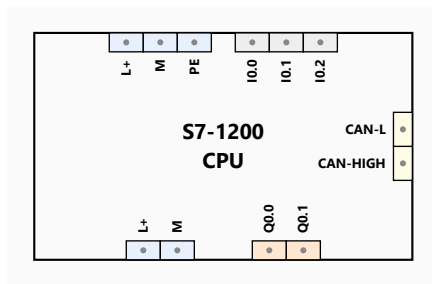
```
#psu(
  "PSU1",
  "rail2.west",
  l_pins: ("L1", "L2", "N"),
  outputs: (("L+", "M"), ("L+", "M"))
)
```

This creates a 3-phase power supply positioned at rail2 with dual output pairs.

2.1.3 Modular Boxes (generic_box)

The foundation for complex modules like PLCs or Gateways.

- **Flexibility:** Allows pin grouping on all four sides.
- **Styling:** Groups can have individual fill colors.
- **Anchors:** Each Anchor can be accessed directly by calling its name (e.g., PLC1.CAN-L for top pin 1).



Visual Example:

Code:

```
#circuit(debug: false, {
  cgeneric_box("PLC1", (5, 0),
    label: "S7-1200 CPU",
    width: 5.0,
    top_pins: (
      ("L+", "M", "PE", (fill: blue.lighten(90%))),
      ("I0.0", "I0.1", "I0.2", (fill: gray.lighten(80%)))
    ),
    bottom_pins: (
      ("L+", "M", (fill: blue.lighten(90%))),
      ("Q0.0", "Q0.1", (fill: orange.lighten(80%)))
    ),
    right_pins: (
      ("CAN-HIGH", "CAN-L", (fill: yellow.lighten(90%)))
    )
  )
})
```

This creates a PLC module with clearly defined input/output groups and color-coded terminals for easy identification.

2.2 Advanced Wiring Techniques

2.2.1 The Projection Principle

To avoid wires crossing through components, use the **Orthogonal Projection** method. This ensures wires run perfectly horizontal from the terminal into the duct.

Syntax: (Point A, "-|", Point B)

- **Effect:** Takes X from Point A and Y from Point B.

Code Example:

```
// Connect CB to vertical duct avoiding crossing
#wire("CB.p1", ("duct-v.east", "-|", "CB.p1"), stroke: black + 1pt)

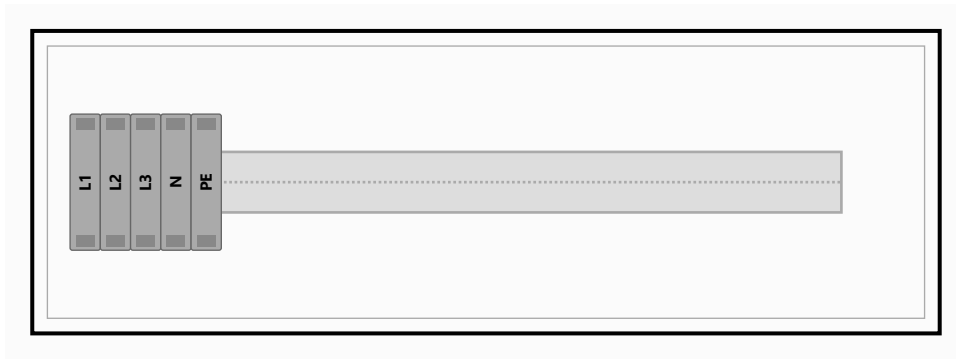
// Multiple orthogonal connections
#wire("F1.p2", ("duct-h.west", "-|", "F1.p2"), stroke: red + 1pt)
```

These examples show how to route wires horizontally from components to ducts.

2.2.2 Terminal Strips (terminal_strip)

Instead of placing terminals manually, use this function to generate an entire numbered block.

- **Mechanism:** It iterates through labels using `range(labels.len())`.



Visual Example:

Code:

```
#circuit(debug: false, {
  cabinet("main", (0, 0), rows: 3, width: 12, height: 4)
  dinrail("rail2", (rel: (0.5, 0), to:("main.slot-2")), length: 10)
  terminal_strip("X1", (rel: (0, 0), to:("rail2.west")), ("L1", "L2", "L3", "N", "PE"))
})
```

2.2.3 Bridging (bridge)

Used for internal jumpers or busbars.

- **Function:** Connects one source to multiple targets with a single common rail.

Code Example:

```
// Power distribution from PSU positive rail
#bridge("PSU.L+", ("K1.11", "K1.A1", "F1.p1"),
  offset: 0.8, stroke: red + 1.5pt)

// Ground busbar connection
#bridge("PSU.M", ("K1.A2", "X1.5", "X2.5"),
  offset: 0.5, stroke: blue + 1.5pt)
```

These examples show distributing power from a PSU to multiple components via busbars.